

AutoSave

Vertigo Save / Load — Implementation Reference

Everything routes through one C++ subsystem: **VTGSave Coordinator**.

0. Get the Coordinator (do this in every graph that saves/loads)

Node: **Get Game Instance Subsystem**

- In the node's **class dropdown**, pick **VTG Save Coordinator**.
- Its output pin (blue object) is the Coordinator. Plug it into the **Target** pin of every save node below.

Get it fresh in each graph. Do NOT promote it to a variable that lives across level loads.

1. Node reference (exact names, pins, return types)

All of these have a **Target** pin = the Coordinator from step 0.

Node	Input pins	Returns
Save To Slot	Slot (int), User Label (string)	bool (success)
Load From Slot	Slot (int)	bool
Auto Save	—	bool

Load Auto Save	—	bool
Has Auto Save	—	bool
Delete Slot	Slot (int)	bool
Does Slot Exist	Slot (int)	bool
Get All Slot Metas	—	Out Metas (array of VTG Slot Meta)
Get Auto Save Slot	—	int (it's 0)
Set Persistent Int	Key (Name), Value (int)	—
Get Persistent Int	Key (Name), Default Value (int)	int
Set Persistent Name	Key (Name), Value (Name)	—
Get Persistent Name	Key (Name)	Name
Clear Persistent Flags	—	—

Bindable events (red, for ISX inventory — see §6): On Save Subsystems (Slot), On Load Subsystems (Slot), On Slot Saved (Slot), On Slot Loaded (Slot).

VTG Slot Meta struct fields (from Get All Slot Metas, for a load menu): Slot Index (int) · Display Label (string) · Stage (Name) · Level Name (string) · Save Time Utc (DateTime) · Save Version (int) · Is Valid (bool — false = empty slot).

2. Variables / components you must create

Where	What to add	Type	SaveGame?	Notes
Player Pawn	Add Component → VTG Player Progress Component	(component)	—	Holds the saved player stats below.
L inside that component	Health	float	☒ (already)	Built in.
L	Max Health	float	☒ (already)	Built in.
L	Counters	Map<Name,int>	☒ (already)	Free-form ints/currencies.
L	Flags	Map<Name,bool>	☒ (already)	Free-form one-off bools.
Player BP	Is Dead?	bool	no	Gates the restart key. You already have this.
Any actor that must persist	the VTG Saveable interface	(interface)	—	Class Settings → Implemented Interfaces.
L that actor's vars	each var to persist	any	☒ you must tick it	Tick SaveGame in the var's Details panel.

The one variable rule: a value is only saved if it lives on a saved object (the Progress Component, or a **VTG Saveable** actor) **and its** `SaveGame` **box is ticked**. Untick = not saved. A var on a random actor with no interface = not saved.

3. Manual save / load

Save: `Get Game Instance Subsystem (VTG Save Coordinator)` → **Save To Slot** (`Slot = 1`, `User Label = "My Save"`).

Load: `Get Game Instance Subsystem (VTG Save Coordinator)` → **Load From Slot** (`Slot = 1`).

4. Checkpoint + death restart

A. Set the checkpoint (e.g. just before a fight): `...Coordinator` → **Auto Save**.

B. Show death screen (player-defeated event in the Player BP):

1. **Create Widget** → `Class = WBP_Death` → output `Return Value`.
2. → **Add to Viewport** (`Target = that Return Value`).
3. **Set** `Is Dead?` = true.

C. "Press R to restart" — INSIDE WBP_Death (not the Player BP):

The R-key event node does NOT fire in widgets. You must use an override.

1. My Blueprint panel → **Functions** → **Override** dropdown → **On Key Down**. This gives a function with `My Geometry` and `In Key Event` inputs and a **Return Node**.
2. Inside it:
 - `In Key Event` → drag → **Get Key** → output `Key`.
 - `Key` → drag → **Equal (Key)** node (the `==`), set its other pin to **R**.
 - → **Branch**.
 - **True** → `Get Game Instance Subsystem (VTG Save Coordinator)` → **Load Auto Save**.
3. On the **Return Node**, set **Return Value** = drag a **Handled** node into it (so the key is marked consumed).
4. Give the widget focus so On Key Down receives keys. On the widget's **Event Construct**:

- **Get Player Controller → Set Input Mode UI Only** (Player Controller pin), and set **In Widget to Focus = Self**.
 - (Optional belt-and-braces: **Set Keyboard Focus**, Target = Self.)
-

5. Resume mid-scene on reload WITHOUT changing Stage

Use a Persistent flag, because changing Stage destroys stage-gated actors, and because a flag is restored **before** the map opens (so BeginPlay can read it). Actor/Progress/Narrative/ISX state is only restored **one tick after** load — too late for a BeginPlay decision.

A. When the phase begins (e.g. OnFinished_CombatIntro), BEFORE **Auto**

Save: ...Coordinator → **Set Persistent Int** (Key = L2StreetFightPhase, Value = 1) → then **Auto Save**.

B. On level start (in BPLM, before your existing intro Branch): ...Coordinator → **Get Persistent Int** (Key = L2StreetFightPhase, Default Value = 0) → **Branch** (Condition: the result == 1)

- **True** → call **EnterCombat** (your custom event that does the combat setup, skips the sequences).
- **False** → play the intro as normal.

Make **EnterCombat** a Custom Event holding your combat-setup nodes (set phase, teleport enemy to front, equip, set view target). Call it from BOTH OnFinished_CombatIntro and the Branch above.

Key strings are arbitrary but must match exactly between Set and Get. Use one per phase (L2StreetFightPhase, L2BarPhase, ...). 1 here just means "combat reached"; use 0/1 like a bool or an enum index for more phases.

6. Inventory (ISX) hookup

In your GameInstance or HUD Blueprint, once:

1. Get Game Instance Subsystem (VTG Save Coordinator).
 2. Drag from it → **Bind Event to On Save Subsystems** → make an event that calls **ISX save**, passing the event's Slot.
 3. Same with **Bind Event to On Load Subsystems** → **ISX load** with Slot.
-

7. Timing & gotchas (the rules that bite)

- **Restored BEFORE map opens** (readable in BeginPlay): the **Stage**, and **Persistent Int/Name** flags.
- **Restored ONE TICK AFTER map loads:** Player Progress Component, **VTG Saveable** actor vars, Narrative quests, ISX inventory. Do not read these in BeginPlay.
- **Stage vs flag:** Stage = "where in the level / what spawns & self-destroys." Persistent flag = "sub-progress within a stage." Use a flag for skip-the-intro.
- **One slot = several files** (VTG_Slot_N, VTG_Manifest_N, VTG_Narrative_N, + ISX) written together. Remove with **Delete Slot**, never by hand.
- **Auto Save = slot 0.** Keep manual saves on slot 1+.